

QUESTION 19

Implement Document Storage and Top-K Retrieval Using a Vector Database

Core Focus: Document Storage, Vector IDs and Top-K Retrieval

Database: FAISS | Index: IndexFlatIP | Top-K: 5

1. Aim

To implement a document retrieval system that stores documents with unique identifiers, maintains their vector representations in a FAISS database, and retrieves the Top-K most relevant documents for a user query.

2. Problem Statement

A vector database returns vector identifiers and similarity scores, but a practical retrieval application must also recover the original document associated with each returned vector. This implementation maintains a document store and an ID-to-document mapping so that the vector IDs returned by FAISS can be converted back into meaningful text.

3. Objectives

- Create a structured document store with unique document IDs.
- Generate vector representations for all stored documents.
- Insert the vectors into a FAISS vector database.
- Search the vector database using a query embedding.
- Retrieve the Top-K vector IDs and similarity scores.
- Map each returned vector ID back to its original document.

4. System Architecture

Top-K Retrieval with Document-ID Mapping

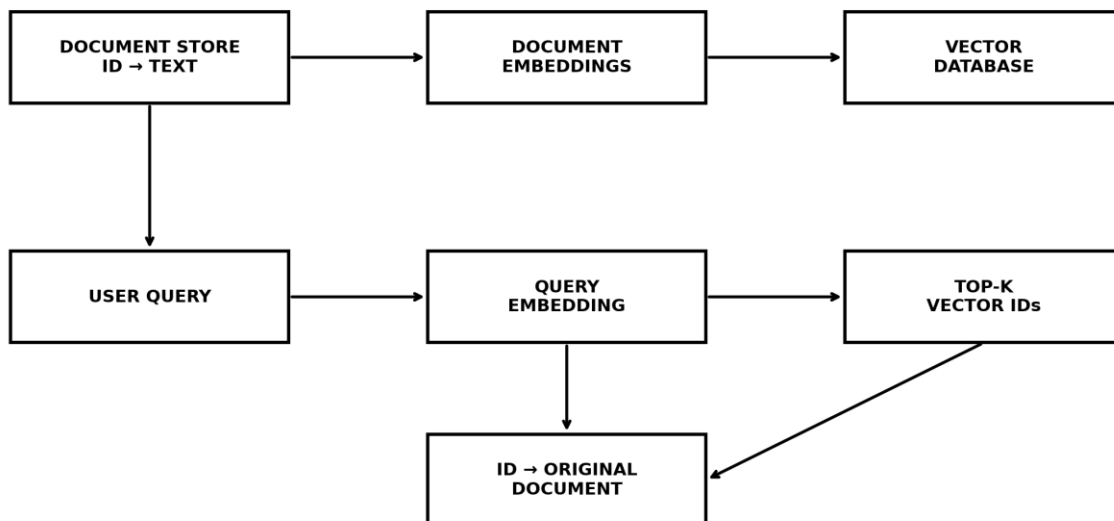


Figure 1: Document storage and vector-ID mapping architecture.

5. Document Storage Model

The implementation uses a dictionary-based document store. Each document is assigned a unique integer ID. The same ID corresponds to the position of the document vector in the FAISS index. When FAISS returns a vector ID, the program uses that ID to access the original document text.

Vector ID	Stored Document Example
0	Machine learning enables computers to learn patterns from data.
1	Artificial intelligence allows machines to perform intelligent tasks.
3	Natural language processing helps computers understand human language.
4	Computer vision enables machines to understand images and videos.
9	Data science uses statistics and machine learning to discover useful patterns.

6. Top-K Retrieval Process

1. Store the document texts with unique IDs.
2. Generate embeddings for all stored documents.
3. Normalize the document vectors.
4. Insert the vectors into the FAISS index.
5. Convert the user query into an embedding.
6. Search the FAISS index for the required K nearest vectors.
7. Receive vector IDs and similarity scores.
8. Use each vector ID to retrieve the corresponding original document.
9. Display the documents in ranked order.

7. Implementation Configuration

Parameter	Actual Value
Database Type	FAISS
Index Type	IndexFlatIP
Stored Documents	10
Stored Vectors	10
Vector Dimension	384
Top-K Value	5
Embedding Model	all-MiniLM-L6-v2

8. Python Implementation

```
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")

document_store = {
    0: "Machine learning enables computers to learn patterns from data.",
    1: "Artificial intelligence allows machines to perform intelligent tasks.",
    2: "Deep learning uses neural networks for advanced data analysis.",
    3: "Natural language processing helps computers understand human language.",
    4: "Computer vision enables machines to understand images and videos.",
    5: "Cloud computing provides scalable computing resources over the internet.",
    6: "Cybersecurity protects computer systems and networks from attacks.",
    7: "Database management systems store and organize structured information.",
    8: "Robotics combines sensors, software, and machines to perform automated tasks.",
    9: "Data science uses statistics and machine learning to discover useful patterns."
}

documents = list(document_store.values())

document_embeddings = model.encode(
    documents, convert_to_numpy=True
).astype("float32")

faiss.normalize_L2(document_embeddings)

dimension = document_embeddings.shape[1]
vector_database = faiss.IndexFlatIP(dimension)
vector_database.add(document_embeddings)

query = input("Enter your search query: ")

query_embedding = model.encode(
    [query], convert_to_numpy=True
).astype("float32")
faiss.normalize_L2(query_embedding)

top_k = 5
scores, vector_ids = vector_database.search(
    query_embedding, top_k
)

for rank, (score, vector_id) in enumerate(
    zip(scores[0], vector_ids[0]), 1
):
    document = document_store[int(vector_id)]
    print(f"Rank {rank}")
    print(f"Vector ID      : {vector_id}")
    print(f"Similarity Score   : {score:.4f}")
    print(f"Retrieved Document : {document}")
```

9. Test Query

How can computers learn from data?

10. Actual Implementation Output

```
TOP-K DOCUMENT RETRIEVAL

Rank 1
Vector ID      : 0
Similarity Score : 0.6549
Retrieved Document : Machine learning enables computers to learn patterns from data.

Rank 2
Vector ID      : 9
Similarity Score : 0.5167
Retrieved Document : Data science uses statistics and machine learning to discover useful patterns.

Rank 3
Vector ID      : 4
Similarity Score : 0.4677
Retrieved Document : Computer vision enables machines to understand images and videos.

Rank 4
Vector ID      : 3
Similarity Score : 0.4490
Retrieved Document : Natural language processing helps computers understand human language.

Rank 5
Vector ID      : 1
Similarity Score : 0.4482
Retrieved Document : Artificial intelligence allows machines to perform intelligent tasks.

VECTOR DATABASE INFORMATION
Database Type   : FAISS
Index Type     : IndexFlatIP
Stored Documents : 10
Stored Vectors  : 10
Vector Dimension : 384
Top-K Value     : 5

BEST RETRIEVED DOCUMENT
Vector ID : 0
Score     : 0.6549
```

Figure 2: Actual Top-K retrieval output from the implemented program.

11. Top-K Retrieval Results

Rank	Vector ID	Score	Retrieved Document
1	0	0.6549	Machine learning enables computers to learn patterns from data.
2	9	0.5167	Data science uses statistics and machine learning to discover useful patterns.
3	4	0.4677	Computer vision enables machines to understand images and videos.
4	3	0.4490	Natural language processing helps computers understand human language.
5	1	0.4482	Artificial intelligence allows machines to

			perform intelligent tasks.
--	--	--	----------------------------

12. Retrieval Score Analysis

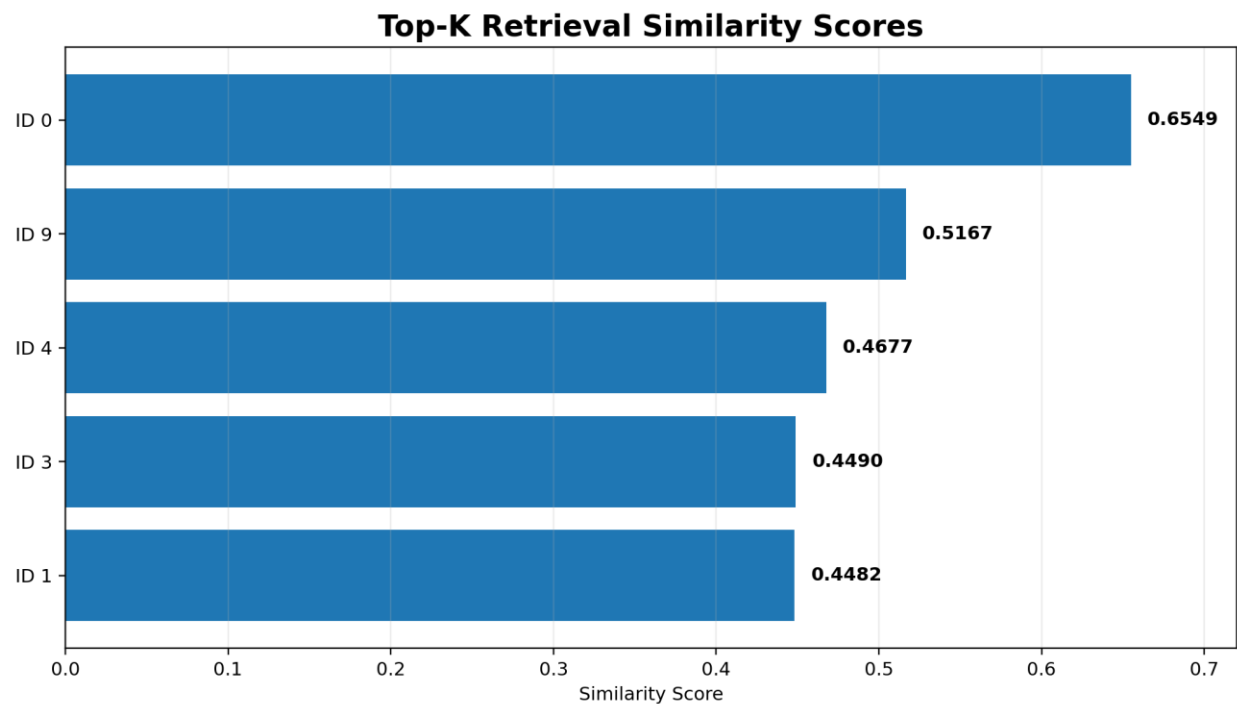


Figure 3: Similarity scores for the five retrieved documents.

The highest-ranked vector is ID 0 with a similarity score of 0.6549. The vector ID is then used to access the corresponding document in the document store. Vector ID 9 is the second result with a score of 0.5167, followed by IDs 4, 3, and 1. This confirms that Top-K retrieval returns both the relevance information and the identifiers required to recover the original content.

13. Result

The document storage and Top-K retrieval system successfully stored 10 documents and 10 corresponding vectors. For the test query, five relevant documents were retrieved. Vector ID 0 was the best match with a similarity score of 0.6549.

14. Advantages

- Maintains a clear relationship between vector IDs and original documents.
- Supports Top-K document retrieval.
- Separates document storage from vector indexing.
- Makes retrieved vector results usable as actual text.
- Provides a practical foundation for RAG retrieval pipelines.

15. Applications

- Document retrieval systems
- Knowledge-base search
- RAG pipelines
- Semantic search applications
- AI assistants
- Enterprise document management

16. Conclusion

A document storage and Top-K retrieval system was successfully implemented using FAISS. Ten documents were stored with unique identifiers and their 384-dimensional embeddings were indexed. The system returned the Top-K vector IDs for the query and mapped those IDs back to the original document texts. The best result was Vector ID 0 with a similarity score of 0.6549. This demonstrates the essential document-mapping mechanism required in practical vector retrieval systems.